

CAN304 Lab 4

CKKS Fully Homomorphic Encryption

Homomorphic Encryption enables computations to be performed directly on encrypted data without decryption, preserving data privacy throughout processing. This lab introduces you to the CKKS (Cheon-Kim-Kim-Song) scheme, which is a practical HE method for approximate arithmetic on real numbers. CKKS is implemented through Microsoft's industrial-grade SEAL library and its accessible Python wrapper, TenSEAL. The primary learning objective is to provide you with hands-on experience in observing HE's homomorphic properties in action: specifically, how encrypted values can be added (additive homomorphism) and multiplied (multiplicative homomorphism) while remaining encrypted, with results matching plaintext computations after decryption.

Dependencies

- Python 3.8+
- Recommended: Virtual environment setup for dependency management
- Linux (Ubuntu/Debian, can be installed on virtual box in the lab computer) or macOS

1. Introduction to Homomorphic Encryption & CKKS

Conceptual Foundation: Traditional encryption creates a fundamental tension between data utility and privacy – you can either keep data encrypted (secure but unusable) or decrypt it for processing (usable but vulnerable). Homomorphic encryption breaks this trade-off by enabling computation on encrypted data while maintaining confidentiality throughout the entire process.

The CKKS Scheme: Unlike exact homomorphic schemes that only handle integers, CKKS supports approximate arithmetic on real and complex numbers through scaled encoding. Numbers are multiplied by a large scale factor before encryption, processed as integers, then scaled back after decryption. This introduces minor computational errors (typically $< 0.001\%$) but dramatically improves performance for real-world applications.

Technology Stack: You'll work with Microsoft's SEAL library via TenSEAL:

- Microsoft SEAL: Industrial-strength FHE implementation in C++ (handles the complex mathematics)
- TenSEAL: Python wrapper that makes SEAL accessible for experimentation and education

Online Resources:

- TenSEAL Documentation: <https://github.com/OpenMined/TenSEAL>
- Microsoft SEAL: <https://github.com/microsoft/SEAL>
- CKKS Explained: <https://blog.openmined.org/ckks-explained-part-1/>

2. Environment Setup (Choose Your Platform)

macOS Setup

```
Bash

# Install Homebrew (if needed)

# Install Python tools
brew install python3 cmake

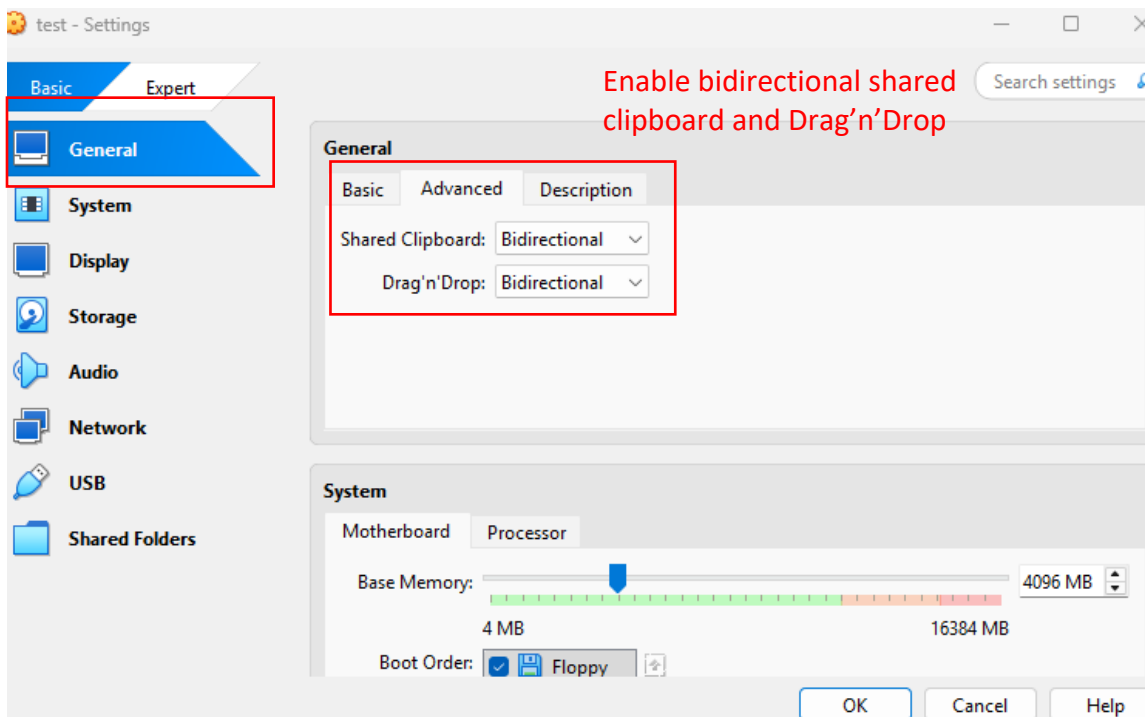
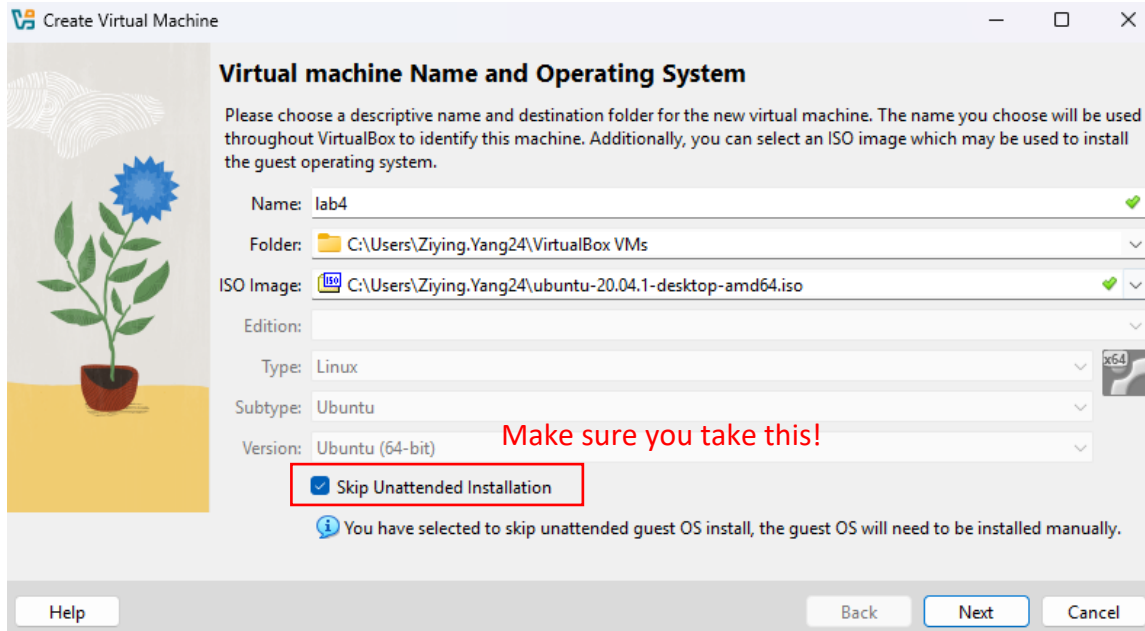
# Setup lab environment
mkdir ckks-lab
cd ckks-lab
python3 -m venv ckks-env
source ckks-env/bin/activate
pip install tenseal numpy matplotlib pandas
```

Linux (Ubuntu/Debian) Setup:

```
Bash

sudo apt update
sudo apt install python3 python3-venv python3-pip cmake build-essential
mkdir ckks-lab
cd ckks-lab
python3 -m venv ckks-env
source ckks-env/bin/activate
pip install tenseal numpy matplotlib pandas
```

NB: In the lab computer, you can find VirtualBox. Install guest OS (Ubuntu 18.04 or later) and use the Linux system to run the experiments. Refer to the screenshot below when installing Ubuntu.



3. Basic Homomorphic Operations

Experiment 1: Setup Verification

First, ensure your environment works correctly with `minimal_test.py`:

Run `minimal_test.py`, you should see:

Ensure ckks-env is activated every time you run the experiments

```
((ckks-env) CV9F4RG2LG:ckks-env zhangjie$ python3 minimal_test.py

CKKS Minimal Test - Verify Your Installation
tenseal version: 0.3.16

1. Creating CKKS context...
✓ Context created successfully

2. Testing encryption/decryption...
Plaintext: [1.5, 2.5, 3.5]
Decrypted: [1.500236, 2.500499, 3.500026]
Decrypted: [1.500236, 2.500499, 3.500026]
Maximum error: 0.0004988384

3. Testing homomorphic addition...
Ciphertext + Ciphertext: [3.000473, 5.000998, 7.000051]
Expected: [3.000000, 5.000000, 7.000000]
Addition works: True

=====
Setup verification: PASSED ✓
You can now proceed to main experiments!
=====
```

Experiment 2: Additive and Multiplicative Homomorphism

Run the main experiment to observe:

$\text{Encrypt}(a) + \text{Encrypt}(b) = \text{Encrypt}(a + b)$

Observe: $\text{Encrypt}(a) \times \text{Encrypt}(b) = \text{Encrypt}(a \times b)$ (element-wise multiplication)

Run `ckks_basic.py`, you should see:

```
[(ckks-env) CV9F4RG2LG:ckks-env zhangjie$ python3 ckks_basic.py

CKKS Homomorphism Observation

Test vectors: A=[1.0, 2.0, 3.0], B=[4.0, 5.0, 6.0]
Test scalar: 2.5

1. Additive Homomorphism:
  Enc([1.0, 2.0, 3.0]) + Enc([4.0, 5.0, 6.0])
  Result: ['5.0000', '7.0000', '9.0000']
  Expected: [5.0, 7.0, 9.0]
  ✓ Works: True

2. Scalar Multiplication:
  Enc([1.0, 2.0, 3.0]) * 2.5
  Result: ['2.5000', '5.0000', '7.5000']
  Expected: [2.5, 5.0, 7.5]
  ✓ Works: True
  Errors: ['0.0000', '0.0000', '0.0000']
  Relative errors: ['0.0%', '0.0%', '0.0%']

3. Integer Scalar Test (for comparison):
  Enc([1.0, 2.0, 3.0]) * 2
  Result: ['2.0000', '4.0000', '6.0000']
  Expected: [2.0, 4.0, 6.0]
  ✓ Works: True
```

4. CKKS Application: Secure Average Age Computation

Three hospitals want to collaborate on research to compute the average age of their combined patients, but they cannot share individual patient records due to privacy regulations (HIPAA/GDPR). Each hospital encrypts their patient data locally using CKKS homomorphic encryption. They send only the encrypted data to a central researcher who can compute the sum and average without ever decrypting individual records. Only the final aggregate result is revealed.

Key Insight: This demonstrates how additive homomorphism ($\text{Enc}(a) + \text{Enc}(b) = \text{Enc}(a + b)$) enables privacy-preserving collaboration in sensitive domains like healthcare.

Run `ckks_app.py`, you should see

```
[(ckks-env) CV9F4RG2LG:ckks-env zhangjie$ python3 ckks_app.py
```

CKKS Application: Secure Average Age Computation

1. Setting up encryption system...

✓ Encryption system ready (Galois keys generated)

2. Simulating hospital data (realistic scenario):

Hospital 1: 26 patients

- Total age: 1375
- Average age: 52.9

Hospital 2: 21 patients

- Total age: 1161
- Average age: 55.3

Hospital 3: 30 patients

- Total age: 1407
- Average age: 46.9

3. Local processing at each hospital:

Hospital 1: ✓ Encrypted 26 patient records

→ Computed encrypted sum

Hospital 2: ✓ Encrypted 21 patient records

→ Computed encrypted sum

Hospital 3: ✓ Encrypted 30 patient records

→ Computed encrypted sum

4. Secure aggregation of hospital sums:

Researcher receives only encrypted sums

Added Hospital 2's encrypted sum

Added Hospital 3's encrypted sum

Final results after decryption:

- Total patients: 77
- Total age sum: 3943.0
- Average age: 51.2

5. Verification (plaintext comparison):

Plaintext calculation:

- Total patients: 77
- Total age sum: 3943
- Average age: 51.2

Accuracy check:

- Total sum error: 0.000 years
- Average age error: 0.000 years
- Error percentage: 0.00%